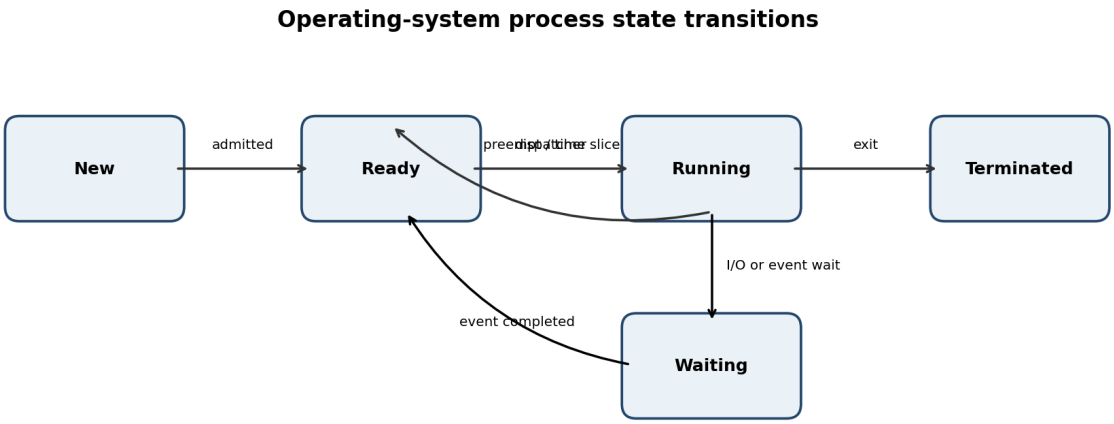
OPERATING SYSTEMS

ASSIGNMENT 2 – CO2

Process Management, Scheduling, Threads, IPC, Synchronisation and Deadlocks



Submitted in partial fulfilment of the Operating Systems course assignment

Student details	Information
Name	S. Maruthu Pandi
Register Number	192511162
Course	CSA0421 – Operating Systems
Assignment	Assignment 2 – CO2
GitHub Repository	https://github.com/Maruthupandi-maker/CSA0421-Operating-System-tree/main/CSA0421%20192511162%20Assignment%202

Q1 – PROCESSES AND PROCESS STATES

1.1 Meaning of a process

A process is a program that is currently executing. It is an active entity with its own process identifier (PID), execution state, program counter, CPU registers, memory mappings, open files and allocated resources. In the online examination server, one process may represent a student session, while other processes perform answer auto-saving, video proctoring, database access and logging.

A process is not the same as a program. A program is a passive file stored on disk; a process is the running instance of that program. Several processes can execute the same program while maintaining separate execution states and address spaces.

1.2 Five process states and transitions

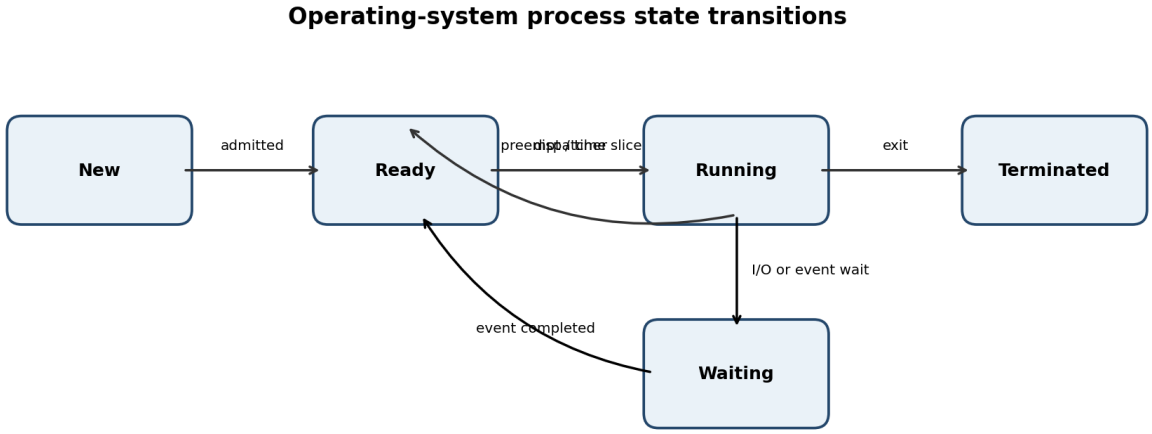


Figure 1. Process-state diagram showing admission, dispatch, pre-emption, waiting and termination.

State	Meaning in the examination server
New	The operating system is creating the student-session process and allocating its initial process structures.
Ready	The process is in main memory and ready to run, but it is waiting for CPU time.
Running	The CPU is currently executing instructions belonging to the process.

Waiting / Blocked	The process cannot continue until an event occurs, such as a disk read, network packet or database response.
Terminated	The process has completed or has been stopped; the OS releases its resources after required cleanup.

A process normally moves from New to Ready after admission. The scheduler dispatches a Ready process to Running. A timer interrupt may pre-empt it and return it to Ready. If it requests I/O, it enters Waiting; when the I/O completes, it returns to Ready. After exit, it enters Terminated.

1.3 Process Control Block (PCB)

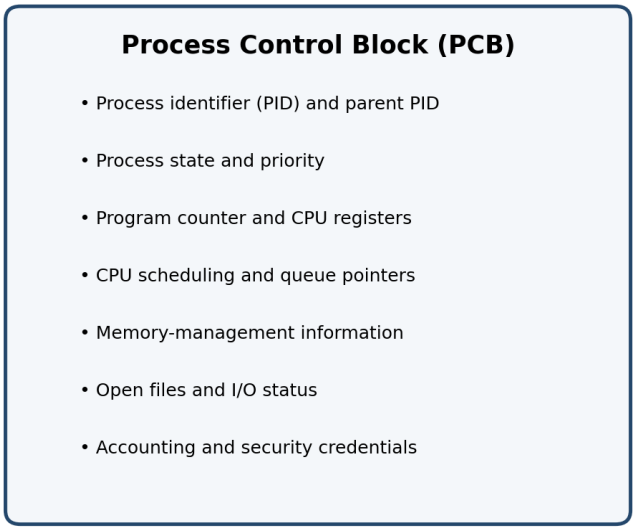


Figure 2. Main fields maintained in a Process Control Block.

The PCB is the kernel data structure that stores the information required to manage and resume a process. During a context switch, the operating system saves the outgoing process state in its PCB and restores the incoming process state from its PCB.

PCB field	Purpose
PID and parent PID	Uniquely identifies the process and records its parent relationship.
Process state and priority	Indicates whether it is ready, running, waiting or terminated and its scheduling importance.
Program counter and registers	Stores the next instruction address and CPU register values so execution can resume.

Scheduling information	Contains priority, time slice information and ready/wait queue links.
Memory information	Records address-space and memory-management details such as page tables.
I/O and open-file information	Tracks open files, devices and pending I/O operations.
Accounting and credentials	Records CPU usage, user identity, permissions and limits.

1.4 Creating child processes with fork()

The fork() system call creates a child process by duplicating the calling process. The child receives a new PID and initially has a logical copy of the parent's address space. Modern Linux systems commonly use copy-on-write (COW), so physical pages are copied only when one process modifies them. fork() returns 0 in the child, the child PID in the parent, and -1 on failure

```c
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <errno.h>

int main(void) {
    pid_t children[3];
    for (int i = 0; i < 3; ++i) {
        pid_t pid = fork();
        if (pid < 0) { perror("fork"); return 1; }
        if (pid == 0) {
            printf("child %d: PID=%ld PPID=%ld\n", i + 1,
                   (long)getpid(), (long)getppid());
            fflush(stdout);
            sleep(10);
            _exit(0);
        }
        children[i] = pid;
    }
    printf("parent PID=%ld created three children\n", (long)getpid());
    fflush(stdout);
    for (int i = 0; i < 3; ++i) {
        int status;
        if (waitpid(children[i], &status, 0) < 0) perror("waitpid");
        else printf("reaped child PID=%ld\n", (long)children[i]);
    }
    return 0;
}
```

Compile and run on Linux:

```
gcc -std=c11 -Wall -Wextra q1_fork.c -o q1_fork
./q1_fork
```

While the children are sleeping, open another terminal and inspect a child PID printed by the program:

```
ps -o pid,ppid,state,stat,comm -p CHILD_PID
cat /proc/CHILD_PID/status
cat /proc/CHILD_PID/stat
```

The ps command displays the PID, parent PID and process state. /proc/PID/status provides readable fields such as Name, State, Pid, PPid, VmRSS and Threads. The process disappears from /proc after it has terminated and been reaped.

1.5 Where time and memory go with 2,000 active processes

Cost factor	What happens at 2,000 sessions	Evaluation
Context switching	The kernel saves and restores registers, scheduler state and memory context when switching among runnable processes.	Too many runnable processes increase scheduling overhead and can disturb cache/TLB locality. Context switches are not free, although they are necessary for responsiveness.
Memory per process	Each process needs a task structure, kernel stack, page tables and its own user-space mappings.	If every session starts a complete server process, duplicated runtime and private memory can consume substantial RAM. COW reduces initial copying but does not eliminate all process overhead.
I/O waiting	Sessions frequently wait for network packets, database queries, disk writes and video input.	A large number of blocked processes can be manageable, but excessive threads/processes and unbounded queues increase memory use and latency.

Burst of logins	2,000 simultaneous logins trigger authentication, TLS, database and session creation work.	Admission control, queues, connection pooling, caching and load balancing are needed to smooth bursts.

1.6 Design decision

Recommended design: use a bounded worker-process pool for fault isolation and a controlled number of worker threads or asynchronous I/O tasks inside each worker. Keep student session state in a shared database or cache rather than making the process itself the only owner of the session. A fixed pool avoids creating 2,000 heavy processes at the same instant, while multiple processes isolate failures better than putting every student in one single process.

Design	Advantages	Disadvantages	Judgement
One process per student	Strong address-space isolation; one crash is less likely to corrupt another session.	Highest process and memory overhead; large login bursts create many process creations and context switches.	Not preferred for 2,000 sessions unless strong isolation is required for a small subset.
One thread per student	Lower creation/switching overhead; threads share code, heap and open resources.	A memory corruption bug can affect the whole server process; shared data needs synchronisation.	Suitable for moderate concurrency with safe thread handling, but requires careful protection.

Fixed worker-process pool	Limits process count; retains process-level isolation; work can be queued and distributed.	Pool size must be tuned; a long job can occupy a worker unless queues and timeouts are used.	Best overall baseline for the examination server; combine with bounded threads or asynchronous I/O.

1.7 Alternative: one thread per student

If each session were a thread, all threads inside a worker process would share the same code, global data, heap, file descriptors and address space. Each thread would have its own stack, registers, program counter, thread-local storage and scheduling state. A thread typically requires much less additional memory than a separate process, although the exact stack reservation is platform- and configuration-dependent.

- Context switching among threads of the same process can be cheaper because the address space is unchanged; switching between processes may also require memory-context and TLB-related work.
- A thread crash caused by an invalid memory write can terminate or corrupt the whole process, affecting many students. Separate worker processes reduce this blast radius.
- The server should therefore use multiple worker processes and a controlled number of threads per worker, rather than a single process containing all sessions.

Complexity comparison: creating and tracking N sessions is O(N) in either model at the application level. The per-session memory constant is generally higher for processes because each has a separate address space and kernel bookkeeping; threads share most process resources. Actual memory and context-switch times must be measured on the target Linux system.